Phase 2 — Grade 1: Linear Algebra (Vectors & Matrices)

Detailed Revision Guide — full formula breakdowns, edge cases, and common-mistake debugging

1. Vectors — The Core Idea

A **vector** is an ordered list of numbers describing ONE 'thing.' Example: a house described by 3 numbers -- [price in lakhs, size in sqft, bedrooms] -> [50, 1200, 3]. ML models only understand numbers, so every 'thing' a model reasons about (a house, image, sentence, customer) gets turned into a vector first -- this is the single most foundational idea in ML.

Analogy that makes it click: a vector is like a fixed-size array, or a simple POJO with numeric fields -- except every field must be a NUMBER, and ORDER MATTERS. [50,1200,3] means something completely different from [1200,50,3] unless the same field order is always kept consistently.

Notation

```
A vector is written as a column (stacked vertically) or informally as a row:
v = [v1, v2, v3, ...]

An arrow or bold letter marks 'this is a vector, not a single number.'

A SCALAR is a single, ordinary number with no direction (e.g. 5, -3, 0.01)
-- contrasted with a vector, which bundles multiple numbers together.
```

1.1 Vector Addition / Subtraction

Formula

```
v1 + v2 = [v1(1)+v2(1), v1(2)+v2(2), ..., v1(n)+v2(n)] -- ELEMENT-BY-ELEMENT

Both vectors MUST be the same length, or the operation is undefined.
```

Worked Example

```
a = [2,4,6], b = [1,1,1]

a + b = [2+1, 4+1, 6+1] = [3, 5, 7]

a - b = [2-1, 4-1, 6-1] = [1, 3, 5]
```

EDGE CASE: Vectors of DIFFERENT lengths cannot be added or subtracted
[2,4,6] + [1,1] is undefined -- there's no matching third element to pair with the '6'. Always check both vectors have the exact same number of elements before adding/subtracting.

1.2 Scalar Multiplication

Formula

```
k × v = [k×v(1), k×v(2), ..., k×v(n)]
```

Multiplying every element by the same number -- stretches/shrinks the vector uniformly. A NEGATIVE scalar also flips the vector's direction, not just its size.

Worked Example

$4 \times [2,4,6] = [4\times2,\ 4\times4,\ 4\times6] = [8,\ 16,\ 24]$

1.3 Magnitude / Length / Norm

Three different NAMES for the exact SAME concept: how 'big' a vector is, geometrically -- the length of the arrow from the origin to the point it represents.

Formula

$$||v|| = \text{sqrt}(\ v(1)^2 + v(2)^2 + \ldots + v(n)^2\ )$$

(an extension of the Pythagorean theorem to more than 2 dimensions)

Worked Example 1: v = [3,4] (classic 3-4-5 triangle)

$||v|| = \text{sqrt}(3^2 + 4^2) = \text{sqrt}(9+16) = \text{sqrt}(25) = 5$

Worked Example 2: v = [6,8]

$||v|| = \text{sqrt}(6^2 + 8^2) = \text{sqrt}(36+64) = \text{sqrt}(100) = 10$

Worked Example 3: c = [2,4,6] (3 dimensions, via np.linalg.norm)

$||c|| = \text{sqrt}(2^2+4^2+6^2) = \text{sqrt}(4+16+36) = \text{sqrt}(56) \approx 7.4833$

COMMON MISTAKE: Writing intermediate calculation steps as if they were VECTOR EQUALITIES
Real example from practice: for v=[6,8], the working was written as 'v = [36,64] = [100] = 10' -- as if the vector [6,8] literally EQUALS the vector [36,64], which then equals [100], which then equals the scalar 10. This is mathematically invalid at every step: a vector can never equal a different vector or a scalar unless they truly contain the same values. What's ACTUALLY happening is a chain of SEPARATE scalar calculations heading toward one final number, not a sequence of equal vectors. Correct notation: $||v|| = \text{sqrt}(6^2 + 8^2) = \text{sqrt}(36+64) = \text{sqrt}(100) = 10$ -- everything under the square root is one running calculation toward a single scalar, never a vector at any intermediate step.

COMMON MISTAKE: Confusing len(vector) with np.linalg.norm(vector) -- the 'length' word collision
English overloads the word 'length' for two UNRELATED things: (1) COUNT of elements -- len(c) for c=[2,4,6] gives 3. (2) GEOMETRIC SIZE -- np.linalg.norm(c) for the same c gives ≈7.483. These measure completely different things and have no numeric relationship to each other. Picture a vector as an arrow on a graph: the norm is the arrow's physical length (as if measured with a ruler); len() just counts how many coordinate numbers define that arrow, which says nothing about how long it actually is.

EDGE CASE: The magnitude of the zero vector is 0
For v=[0,0,0], ||v|| = sqrt(0+0+0) = 0 -- the only vector with zero magnitude is the all-zeros vector (the one sitting exactly at the origin, with no length in any direction).

1.4 Dot Product

Takes two vectors of the SAME LENGTH and produces ONE scalar: multiply corresponding elements, then sum. This is the single most important vector operation in ML -- it is the EXACT computation every neuron in a neural network performs on its inputs and weights.

Formula

```
a · b = a(1)×b(1) + a(2)×b(2) + ... + a(n)×b(n)
```

Worked Example 1: a=[2,4,6], b=[1,0,1]

```
a · b = (2×1) + (4×0) + (6×1) = 2 + 0 + 6 = 8
```

Worked Example 2: p=[3,6,9], q=[1,0,-1]

```
p · q = (3×1) + (6×0) + (9×-1) = 3 + 0 - 9 = -6 (dot products CAN be
negative)
```

EDGE CASE: The dot product requires EQUAL-LENGTH vectors; mismatched lengths = undefined
[2,4,6]·[1,0] is undefined -- there's no element in the second vector to pair with the first vector's third entry. This exact requirement (equal length) is WHY matrix multiplication later requires matching 'inner dimensions' -- see Section 3.

TIP: Forward connection: a neuron's computation IS a dot product
inputs=[x1,x2,x3], weights=[w1,w2,w3] -> a neuron computes dot(inputs,weights)+bias. This single number then decides how 'activated' that neuron is. Full depth on this is covered properly in a later neural-network grade -- for now, just recognize the dot product is not an abstract exercise.

2. Matrices — The Core Idea

A **matrix** is multiple vectors STACKED TOGETHER -- a grid of rows and columns describing MANY things at once. If a vector describes ONE house, a matrix describes MANY houses, stacked as rows.

Example matrix (3 houses, 3 features each)

```
House A: 50, 1200, 3
House B: 80, 1800, 4
House C: 30, 900, 2
```

Analogy that makes it click: this is exactly a 2-D array (like Java's int[][]) or a plain table with no column names. Each ROW = one record (one house). Each COLUMN = one feature shared across all records (all prices in column 1, all sizes in column 2, etc.).

2.1 Dimensions

Formula

```
Dimensions are written ROWS × COLUMNS, always rows first.
3 houses, 3 features -> 3×3. 4 students, 2 test scores each -> 4×2.
(Same convention as NumPy's .shape -- rows first, columns second.)
```

2.2 Matrix Indexing

Formula (identical syntax to NumPy 2-D arrays from Phase 1)

```
matrix[row, col] -> a single element
matrix[row, :] -> that entire row (all columns)
matrix[:, col] -> that entire column (all rows)
```

Worked Example: matrix = [[80,90],[70,60],[95,85],[60,70]] (4 students x 2 test scores)

```
matrix[2, :] -> [95, 85] (Student at index 2 -- the THIRD student, since zero-indexed)
matrix[:, 1] -> [90, 60, 85, 70] (everyone's score at column index 1)
matrix[0, 1] -> 90 (single value: row index 0, column index 1)
```

EDGE CASE: Zero-indexing applies here exactly like NumPy arrays and Python lists
'Row 2' always means INDEX 2 (the third physical row), not 'the second row by counting.' This exact ambiguity caused confusion earlier in the curriculum (NumPy/Pandas grades) -- the rule is consistent everywhere: always zero-indexed unless explicitly told otherwise.

2.3 Matrix Multiplication

Every entry in the result is the DOT PRODUCT of one row from the first matrix and one column from the second. This directly reuses the dot product from Section 1.4 -- it's literally 'many dot products, computed systematically, all at once.'

Validity Rule

```
A (m × n) × B (p × q) is only valid if n = p ('inner dimensions must match')
Result size = (m × q) ('outer dimensions')

WHY: each result entry is a dot product of one row of A and one column of B. A dot product needs two vectors of EQUAL length (Section 1.4's edge case). A row of A has length n (A's column count); a column of B has length p (B's row count). So n and p must match, or there's nothing valid to pair up.
```

Validity Check Examples

```
3×4 times 4×2 -> inner dims 4 and 4 MATCH -> VALID, result is 3×2
2×3 times 4×5 -> inner dims 3 and 4 DO NOT MATCH -> INVALID, cannot multiply
```

Worked Example 1: A=[[1,2],[3,4]], B=[[5,6],[7,8]] (both 2×2)

```
Result[0,0] = row0(A)·col0(B) = (1×5)+(2×7) = 5+14 = 19
Result[0,1] = row0(A)·col1(B) = (1×6)+(2×8) = 6+16 = 22
Result[1,0] = row1(A)·col0(B) = (3×5)+(4×7) = 15+28 = 43
Result[1,1] = row1(A)·col1(B) = (3×6)+(4×8) = 18+32 = 50
```

```
A × B = [[19,22],[43,50]]
```

Worked Example 2: A=[[2,0],[1,3]], B=[[4,1],[2,5]]

```
Result[0,0] = (2×4)+(0×2) = 8+0 = 8
Result[0,1] = (2×1)+(0×5) = 2+0 = 2
Result[1,0] = (1×4)+(3×2) = 4+6 = 10
Result[1,1] = (1×1)+(3×5) = 1+15 = 16
A × B = [[8,2],[10,16]]
```

COMMON MISTAKE: Pairing the WRONG row-element with the WRONG column-element inside a dot product

Real example from practice: for P=[[1,2],[3,4]], Q=[[2,0],[1,2]], finding Result[1,1] (row1(P)·col1(Q) -- row1(P)=[3,4], col1(Q)=[0,2]) was first written as '(4×0 + 3×2)' -- swapping which P-value multiplies with which Q-value. The CORRECT pairing is always 'first element of the row with first element of the column, second with second': (3×0) + (4×2), not (4×0)+(3×2) -- even though both use the same four numbers, only the first pairing is the actual dot product. **Debug habit:** write the row and column out explicitly as two separate little lists first (row1(P)=[3,4], col1(Q)=[0,2]), then pair strictly by POSITION (1st-with-1st, 2nd-with-2nd) before multiplying anything.

COMMON MISTAKE: Simple addition slip after the multiplication is already correct

Continuing the same example: even after correctly writing '(3×0) + (4×2)', the final sum was given as 6 instead of the correct 8 (0+8=8, not 6). The multiplication-pairing was right, but the final addition itself was rushed. **Debug habit:** after a multi-term calculation, re-add the final numbers slowly and separately as a last dedicated check, the same way you'd double-check a power-rule evaluation's final multiplication (see Grade 2's revision guide).

*The * vs @ distinction in NumPy*

```
A * B -> WRONG for matrix multiplication -- this does ELEMENTWISE multiply (position-by-position)
A @ B -> CORRECT -- true matrix multiplication (row·column dot products)
np.matmul(A, B) -> identical to A @ B, just a named function
```

COMMON MISTAKE: Using * instead of @ for matrix multiplication (a very common beginner trap, flagged generally)

A*B multiplies element [0,0] of A with element [0,0] of B, [0,1] with [0,1], and so on -- a COMPLETELY different operation from true matrix multiplication. For A=[[2,0],[1,3]], B=[[4,1],[2,5]]: A*B gives [[8,0],[2,15]], while A@B correctly gives [[8,2],[10,16]]. These look superficially similar (both involve the same two matrices) but are mathematically unrelated operations -- always use @ (or np.matmul) when true matrix multiplication is intended.

2.4 Order Matters: A × B ≠ B × A

Unlike plain number multiplication (where 3×5 = 5×3), matrix multiplication is generally NOT commutative -- swapping the order changes the result, or can make a previously-valid multiplication invalid entirely (e.g. a 3×4 times a 4×2 is valid, but the REVERSE, 4×2 times 3×4, has inner dimensions 2 and 3, which don't match -- so B×A isn't even defined here). Always compute in the exact order specified in the problem.

EDGE CASE: A valid A×B does not guarantee B×A is even DEFINED
Check both directions' inner dimensions SEPARATELY before assuming you can just flip the order. A valid m×n times n×q only guarantees the FORWARD direction works; the reverse (n×q times m×n, which really means q×n-shaped on the left... requires its own separate dimension check) may be entirely invalid.

2.5 Forward Connection: Matrix Multiplication and Neural Network Layers

One neuron computes ONE dot product (inputs·weights + bias) -- one number out. A full network LAYER has MANY neurons at once, each with its own weight vector. Instead of computing each neuron's dot product one at a time, all the neurons' weights are stacked into a matrix, and ONE matrix multiplication computes EVERY neuron's output simultaneously. This is the entire reason matrix multiplication is the computational core of deep learning: it is 'many dot products, done all at once, efficiently.' (Full depth with real weights/bias/activation functions is covered properly in the dedicated neural-network grade later in the curriculum.)

3. Full Debugging Checklist for This Grade

---

1. Before adding/subtracting vectors, are they the SAME length? 2. Am I keeping vector equalities and scalar calculations separate (never writing 'vector = scalar' or 'vector = different vector')? 3. Am I confusing len() (count) with norm() (geometric magnitude)? 4. Before multiplying two matrices, do the INNER dimensions actually match (A's columns = B's rows)? 5. When computing a dot product inside a matrix multiply, have I written the row and column out as two explicit lists FIRST, then paired strictly by position? 6. Have I double-checked the FINAL addition in a multi-term dot product, separately from the multiplication steps? 7. Am I using @ (or np.matmul), not *, whenever TRUE matrix multiplication is intended? 8. Have I checked dimensions separately for A×B and B×A, rather than assuming one validity implies the other?